MPI простыми словами: как это работает и как писать код

22 окт. 2026 г. · @NeoCoder

1. Что такое MPI и главная идея

MPI (Message Passing Interface) это набор функций, который позволяет запустить несколько копий одной программы и дать им обмениваться данными. Всё остальное в лабораторной вырастает из этой идеи.

Представь четырёх человек в четырёх разных комнатах. У всех одна и та же инструкция, но у каждого на двери табличка с номером: 0, 1, 2, 3. Видеть друг друга они не могут и общаются только записками. Это и есть MPI.

- **Процесс**: одна запущенная копия твоей программы. Команда `mpiexec -np 4 программа` запускает 4 процесса.
- **Ранг**: номер процесса (0, 1, 2, 3). Именно по рангу процессы понимают, кто они и что им делать.
- **Память у каждого своя**: переменная `x` в процессе 0 и переменная `x` в процессе 1 это две разные переменные. Поменять чужую память нельзя, можно только отправить сообщение.
- **Процессы работают одновременно**: поэтому строки в консоли приходят в случайном порядке. Это нормально, а не ошибка.

Из этого следует главный принцип: код пишется один, а процессы расходятся по разным веткам `if (rank == ...)`.

2. Скелет любой MPI-программы

Каждая программа из лабораторной построена на одном и том же каркасе из четырёх вызовов:

```
#include <mpi.h>

int main(int argc, char** argv)
{
    int rank, size;

    MPI_Init(&argc, &argv);                  // 1. включаем MPI
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);    // 2. узнаём свой номер
    MPI_Comm_size(MPI_COMM_WORLD, &size);    // 3. узнаём, сколько всего
процессов

    // ... здесь работа программы ...

    MPI_Finalize();                          // 4. выключаем MPI
    return 0;
}
```

Что делает каждая строка:

- `MPI_Init` запускает MPI. Без неё ни одна другая функция MPI работать не будет.
- `MPI_COMM_WORLD` это «общий чат» всех процессов программы. Его создаёт MPI сам, и почти везде в лабораторной ты будешь просто писать это имя.
- `MPI_Comm_rank` записывает в `rank` номер текущего процесса.
- `MPI_Comm_size` записывает в `size` общее число процессов (то самое число из `-np`).
- `MPI_Finalize` завершает работу MPI. После неё вызывать функции MPI нельзя.

Все числа передаются в функции через `&`, потому что функция должна записать результат в твою переменную.

3. Как разделить роли: if (rank == ...)

Все процессы выполняют один и тот же код, но каждый знает свой ранг. Поэтому роли распределяются обычным `if`:

```
if (rank == 0)
{
    // этот код выполнит только процесс 0 (например, «главный»)
}
else
{
    // этот код выполнят все остальные процессы («рабочие»)
}
```

Типичные роли в заданиях лабораторной:

- **Процесс 0 главный**: читает данные с клавиатуры, раздаёт работу, собирает ответы, печатает итог.
- **Остальные рабочие**: получают кусок работы, считают, отправляют результат главному.
- **Чётные и нечётные**: `if (rank % 2 == 0)` делит процессы на две группы (задание 2).

Важное правило: ввод с клавиатуры (`cin`) под `mpiexec` получает только процесс 0. Если остальным нужно это число, процесс 0 должен отправить его им сообщением (про это следующий раздел).

Ещё одно следствие: раз процессы идут по разным веткам, они почти всегда выполняют разное число вызовов MPI. Поэтому каждому `MPI_Send` в одной ветке должен соответствовать `MPI_Recv` в другой.

4. Отправка и приём: MPI_Send и MPI_Recv

Сообщение в MPI это «письмо»: что отправить, сколько, кому и с какой пометкой. Отправитель вызывает `MPI_Send`, получатель вызывает `MPI_Recv`.

```
int x = 42;

if (rank == 1)
    MPI_Send(&x, 1, MPI_INT, 0, 5, MPI_COMM_WORLD);   // процесс 1 отправляет
число процессу 0

if (rank == 0)
{
```

```
    int y;
    MPI_Status st;
    MPI_Recv(&y, 1, MPI_INT, 1, 5, MPI_COMM_WORLD, &st);  // процесс 0
получает его от процесса 1
}
```

Что означает каждый параметр:

Параметр	Смысл простыми словами	В примере
`buf`	адрес переменной (или массива), откуда берём или куда кладём данные	`&x`
`count`	сколько элементов отправляем (в `Recv`: максимум, сколько готовы принять)	`1`
`type`	тип одного элемента в терминах MPI	`MPI_INT`
`dest` / `source`	ранг получателя (в `Send`) или отправителя (в `Recv`)	`0` / `1`
`tag`	число-пометка, чтобы различать сообщения	`5`
`comm`	в каком «общем чате» общаемся	`MPI_COMM_WORLD`
`status`	(только `Recv`) сюда MPI запишет, кто и что прислал	`&st`

Типы данных MPI соответствуют обычным типам C++:

В C++	В MPI
`char`	`MPI_CHAR`
`int`	`MPI_INT`
`long long`	`MPI_LONG_LONG`
`float`	`MPI_FLOAT`
`double`	`MPI_DOUBLE`

Правило пары: `Send` и `Recv` должны совпадать по тегу, коммуникатору и типу, а отправитель и получатель должны указывать друг друга. Если `Recv` ждёт то, что никто не отправляет, программа зависнет и будет ждать вечно.

Массив отправляется тем же вызовом: передаёшь адрес первого элемента и количество. Для `vector` это `v.data()` и `v.size()`.

5. Когда не знаешь, кто прислал: ANY_SOURCE, ANY_TAG, status

Иногда главный процесс ждёт ответы от рабочих, но не знает, кто ответит первым. Тогда вместо ранга отправителя пишут `MPI_ANY_SOURCE` («от кого угодно»), а вместо тега `MPI_ANY_TAG` («с любым тегом»).

Кто на самом деле прислал, можно узнать из переменной `status`:

```
MPI_Status st;
long long part;

MPI_Recv(&part, 1, MPI_LONG_LONG, MPI_ANY_SOURCE, MPI_ANY_TAG,
MPI_COMM_WORLD, &st);
```

```
int who = st.MPI_SOURCE;    // ранг того, кто прислал
int tag = st.MPI_TAG;       // тег этого сообщения
```

Сколько элементов реально пришло (если буфер был с запасом), покажет `MPI_Get_count`:

```
int n;
MPI_Get_count(&st, MPI_INT, &n);    // n = число полученных элементов
```

Это главный инструмент задания 5: главный процесс принимает ответ от любого рабочего через `MPI_ANY_SOURCE`, смотрит в `st.MPI_SOURCE`, кто освободился, и отправляет именно ему новую работу.

6. Блокирующие обмены и тупик (deadlock)

`MPI_Send` и `MPI_Recv` блокирующие: вызов не возвращает управление, пока операция не дойдёт до безопасной точки. `Recv` ждёт, пока сообщение придёт. `Send` ждёт, пока его данные можно будет не трогать (либо они скопированы во внутренний буфер MPI, либо получатель начал приём).

Отсюда тупик. Два процесса хотят обменяться данными и оба сначала вызывают `Send`:

```
// процесс 0                        // процесс 1
MPI_Send(... 1 ...);                MPI_Send(... 0 ...);
MPI_Recv(... 1 ...);                MPI_Recv(... 0 ...);
```

Каждый застрял в `Send` и ждёт, пока другой начнёт `Recv`. Но до `Recv` никто не дойдёт, так как оба стоят в `Send`. Так вечно и висят.

Почему с маленькими сообщениями такой код иногда работает, а с большими нет: маленькое сообщение MPI может молча скопировать в свой буфер, и `Send` сразу вернётся. Большое (в задании 3 это 32768 чисел float, около 128 КБ) в буфер не помещается, и `Send` честно ждёт получателя. Поэтому код, который «работает», может быть ошибочным.

Три способа избежать тупика:

1. **Развести порядок**: процесс 0 сначала отправляет, потом принимает, а процесс 1 сначала принимает, потом отправляет.
2. **`MPI_Sendrecv`**: одна функция делает и отправку, и приём, а MPI сам следит, чтобы не было тупика.
3. **Неблокирующие вызовы** (раздел 8). Этого требует задание 3.

Вот `MPI_Sendrecv` для обмена с соседом:

```
MPI_Sendrecv(&mine,   1, MPI_INT, other, 0,    // что отправляем, кому, с каким тегом
             &theirs, 1, MPI_INT, other, 0,    // куда принимаем, от кого, с каким тегом
             MPI_COMM_WORLD, &st);
```

У неё есть вариант `MPI_Sendrecv_replace`: отправляет и принимает через один и тот же буфер, поэтому полученное затирает отправленное.

7. Четыре режима отправки

Отправить сообщение можно четырьмя способами. Они различаются тем, когда `Send` считается завершённым. Принимают всегда одним и тем же `MPI_Recv`: режим выбирает только отправитель.

Режим	Функция	Когда вызов завершится	Условие
Стандартный	`MPI_Send`	MPI решает сам: или сразу (скопировал в буфер), или когда получатель начал приём	нет
Синхронный	`MPI_Ssend`	только когда получатель начал `Recv`	нет
Буферизованный	`MPI_Bsend`	сразу: данные копируются в твой буфер	нужно заранее выдать MPI буфер
По готовности	`MPI_Rsend`	сразу	`Recv` получателя **уже** должен ждать, иначе ошибка

Как это понять на примере с почтой: `Send` это бросить письмо в ящик и пойти дальше, если ящик не переполнен. `Ssend` это передать письмо лично в руки и дождаться расписки. `Bsend` это сначала положить копию в свою папку «исходящие», а доставкой займётся почтальон. `Rsend` это крикнуть «лови!», когда человек уже стоит с протянутыми руками. Если его там нет, письмо потеряется (в MPI это ошибка).

Буферизованный режим требует лишних шагов:

```
int size = sizeof(int) + MPI_BSEND_OVERHEAD;   // место под сообщение + служебная часть
char* buf = new char[size];

MPI_Buffer_attach(buf, size);                  // отдаём буфер MPI
MPI_Bsend(&num, 1, MPI_INT, 1, 100, MPI_COMM_WORLD);
MPI_Buffer_detach(&buf, &size);                // забираем буфер обратно (ждёт отправки)
```

На практике почти всегда используют обычный `MPI_Send`. Остальные режимы нужны для особых случаев и для понимания, как устроена передача.

8. Неблокирующие обмены: Isend, Irecv, Wait, Test

У каждой парной функции есть неблокирующая версия с буквой `I` (Immediate, «немедленно»). Она только **запускает** передачу и сразу возвращает управление. Программа идёт дальше, а пересылка идёт «на фоне».

Аналогия: блокирующий `Send` это стоять у почтового окошка, пока письмо не примут. Неблокирующий `Isend` это отдать письмо и пойти заниматься делами, а потом вернуться и проверить, ушло ли оно.

У любой неблокирующей операции две стадии:

1. **Запуск**: `MPI_Isend` или `MPI_Irecv`. Они выдают «квитанцию» типа `MPI_Request`.
2. **Проверка завершения**: `MPI_Wait` (ждать, пока закончится) или `MPI_Test` (просто проверить и идти дальше).

```
MPI_Request reqs[2];
```

```
MPI_Isend(sendbuf, n, MPI_FLOAT, other, tag1, MPI_COMM_WORLD, &reqs[0]);   // запустили отправку
MPI_Irecv(recvbuf, n, MPI_FLOAT, other, tag2, MPI_COMM_WORLD, &reqs[1]);   // запустили приём

// здесь можно считать что-то, что не трогает sendbuf и recvbuf

MPI_Waitall(2, reqs, MPI_STATUSES_IGNORE);   // ждём, пока закончатся обе операции
// только теперь recvbuf можно читать, а sendbuf менять
```

Важнейшее правило: **пока не вызван `Wait` (или `Test` не сказал «готово»), буферы трогать нельзя**. Отправляемый массив нельзя менять, принимаемый нельзя читать: там может быть мусор или половина данных.

Функции ожидания:

Функция	Что делает
`MPI_Wait`	ждёт одну операцию
`MPI_Waitall`	ждёт все операции из массива
`MPI_Waitany`	ждёт, пока закончится любая одна, и говорит, какая
`MPI_Waitsome`	ждёт, пока закончится хотя бы одна, и говорит, какие
`MPI_Test`	не ждёт: записывает в `flag`, готово ли (1 или 0)

Почему это полезно: не бывает тупика, как в разделе 6 (оба процесса сначала запускают обмен, а потом ждут), и можно считать, пока идёт пересылка. Но ответственность за правильную работу с буфером теперь на тебе.

9. Замер времени: MPI_Wtime

`MPI_Wtime()` возвращает число секунд (типа `double`), прошедших с какого-то момента в прошлом. Сам момент отсчёта неважен: важна разница между двумя вызовами.

```
double start = MPI_Wtime();

// ... то, что хотим измерить ...

double end = MPI_Wtime();
std::cout << end - start << std::endl;   // время в секундах
```

Пара советов для задания 5, где нужно сравнить быстродействие:

- Замеряй время на процессе 0 вокруг всей работы целиком (от раздачи заданий до получения последнего ответа), а не вокруг одного сообщения.
- Запускай в конфигурации **Release**: в Debug программа заметно медленнее, и сравнение получится искажённым.
- Запускай каждую пару N и M несколько раз и бери типичное значение: время немного «прыгает».
- `MPI_Wtick()` показывает точность таймера (минимальный шаг в секундах).

10. Как думать о заданиях лабораторной

Для каждого задания полезно сначала ответить на два вопроса: **кто какую роль играет** и **кто кому что отправляет**. Код после этого пишется почти сам.

Задание 1: вывести свой номер и число процессов. Здесь нет обмена сообщениями. Каждый процесс вызывает `MPI_Comm_rank` и `MPI_Comm_size` и печатает результат. Порядок строк в консоли случайный.

Задание 2: чётные и нечётные. Роли делятся по `rank % 2`: чётные печатают одну строку, нечётные другую. Дополнительно процесс 0 печатает `size`.

Задание 3: найти и вылечить тупик. Оба процесса сначала вызывают `MPI_Send` на большом сообщении и оба зависают (раздел 6). Лечение: заменить `MPI_Send` и `MPI_Recv` на `MPI_Isend` и `MPI_Irecv` и дождаться их через `MPI_Waitall`. В отчёте объясни, почему был тупик и почему теперь его нет.

Задание 4: эстафетная палочка. Данные идут по кругу:

- процесс 0 генерирует число и отправляет процессу 1;
- каждый следующий процесс принимает от предыдущего (`rank - 1`), прибавляет 1 и отправляет следующему;
- последний процесс отправляет число обратно процессу 0, а следующего для него считают так: `(rank + 1) % size`;
- процесс 0 принимает от последнего и сверяет: итог должен быть на `size - 1` больше исходного.

Если число прошло через `size - 1` процессов, каждый из которых прибавил 1, получается именно `size - 1`.

Задание 5: главный и рабочие (master-slave). Это самое объёмное задание. Логика такая:

1. Главный (процесс 0) отправляет каждому рабочему по одному массиву.
2. Рабочий считает сумму массива и отправляет число обратно.
3. Главный принимает ответ от любого рабочего (`MPI_ANY_SOURCE`), прибавляет к общей сумме и по `st.MPI_SOURCE` узнаёт, кто освободился.
4. Если массивы ещё остались, главный отправляет освободившемуся новый. Если нет, отправляет «стоп».
5. Рабочий в цикле принимает работу, пока не придёт «стоп». Сигнал «стоп» удобно передавать особым тегом (`status.MPI_TAG`).

Пишется два варианта: с блокирующими вызовами и с неблокирующими. В выводах сравни время для трёх пар N и M: при большом M передаётся мало больших сообщений, при большом N много маленьких, и накладные расходы на пересылку влияют по-разному.

Задание 6: транспонирование «каждый каждому». На процессе `i` лежит строка `i` матрицы. После обмена на процессе `j` должен оказаться столбец `j`. Значит, элемент номер `j` строки процесса `i` должен уехать процессу `j`, а от процесса `j` процесс `i` должен получить его элемент номер `i`. То есть каждая пара процессов обменивается ровно одним числом. Для этого удобно `MPI_Sendrecv` в цикле по всем процессам, кроме самого себя: он не даёт повиснуть в тупике.

11. Шпаргалка и типичные ошибки

Что нужно	Функция
Включить и выключить MPI	`MPI_Init`, `MPI_Finalize`
Узнать свой номер и число процессов	`MPI_Comm_rank`, `MPI_Comm_size`
Отправить и принять (блокирующе)	`MPI_Send`, `MPI_Recv`
Обменяться без тупика	`MPI_Sendrecv`, `MPI_Sendrecv_replace`
Отправить и принять (неблокирующе)	`MPI_Isend`, `MPI_Irecv`
Дождаться неблокирующей операции	`MPI_Wait`, `MPI_Waitall`, `MPI_Waitany`
Проверить, готова ли операция	`MPI_Test`
Принять от любого	`MPI_ANY_SOURCE`, `MPI_ANY_TAG`
Узнать, кто и сколько прислал	`status.MPI_SOURCE`, `status.MPI_TAG`, `MPI_Get_count`
Замерить время	`MPI_Wtime`

Частые ошибки и как их узнать:

- **Программа зависла и ничего не выводит.** Чаще всего `Recv` ждёт сообщение, которое никто не отправляет (не совпал ранг или тег), либо двойной `Send` до `Recv` (тупик). Проверь, что у каждого `Send` есть парный `Recv`.
- **Ошибка `MPI_ERR_TRUNCATE`.** Сообщение пришло больше, чем размер буфера в `Recv`. Увеличь `count` в `Recv`.
- **Неверный тип.** Для `long long` нужен `MPI_LONG_LONG`, а не `MPI_INT`. Тип в `Send` и `Recv` должен совпадать с типом переменной.
- **В программе только один процесс (`size` равно 1).** Запустили без `mpiexec`. Запускай с `-np`.
- **Данные в неблокирующем обмене испорчены.** Прочитал или изменил буфер до `Wait`.
- **Забыт `MPI_Finalize` или вызов MPI после неё.** Программа может завершиться с ошибкой.
- **Программа не запускается или не собирается.** Проверь, что платформа x64, пути к `mpi.h` и `msmpi.lib` указаны, а в пути к проекту нет русских букв.
- **Остальные процессы не получили число, введённое с клавиатуры.** `cin` читает только процесс 0. Остальным число нужно отправить сообщением.
- **Строки выводятся вперемешку.** Это нормально: процессы работают одновременно. Если порядок важен, пусть печатает один процесс, собрав данные от остальных.

Правило, которое решает половину проблем: для каждого сообщения всегда можно ответить на вопросы «кто отправляет, кто принимает, какой тег, какой тип, какой размер». Если на какой-то из них ответ не очевиден, ошибка скорее всего там.